

Distribution kit

Mine-FNL

Distribution kit

Pre-written launch artifacts for the public release of firm-bot. Drop these into the matching channels; tweak voice for the audience.

Show HN post

Title: firm-bot – local-first chatbot for legal/audit/consultancy docs

Body:

I built a multi-tenant, citation-required chatbot that runs entirely on the firm's own hardware — Ollama + Chroma + BM25 in a single Python package. No cloud calls, no telemetry, no API keys to manage.

The wedge is that lawyers can paste the bot's output into client emails because every factual claim carries a `[file:page]` citation marker and a second LLM-as-judge flags anything that's not grounded in the retrieved sources. This is the only RAG system I've seen where the honest answer to "why did you say that?" is always one click away.

What ships in v0.1:

- **PDF / EML / DOCX ingestion** with structure-aware chunking that recognises Article / Section / standalone Title Case headings like "Term" and "Governing Law" — naive sliding-window chunkers lose these clause boundaries.
- **Hybrid retrieval** — BM25 + dense embeddings fused via Reciprocal Rank Fusion, plus an optional cross-encoder reranker for hard cross-document questions.
- **Per-firm isolation** — each client gets its own Chroma collection, its own BM25 pickle, its own filesystem root. No shared index.
- **Citation-required prompts** + a 7B judge model that flags any claim without a `[file:page]` marker.
- **PII redaction at ingest** — SSN, EIN, email, phone, credit card, IBAN, IPv4 replaced with stable category tokens before chunks are made.
- **SSE streaming** — tokens render live in the web UI.
- **Eval harness** with real public benchmarks against CUAD (Contract Understanding Atticus Dataset) — the industry standard for legal contract QA.

Measured numbers on a 10-contract CUAD subset: firm-bot hits 53.8% precision@k vs 46.2% for naive chunking — a 7.6 pp lift that comes entirely from structure-aware chunking. End-to-end

benchmark with the full RAG pipeline (retrieval + LLM + guard): 100% citation coverage, 75% keyword coverage, sub-2s p50 latency on a 7B model on an M4.

The whole thing fits in `pip install firm-bot + docker compose up`. One command, no API keys, no signup, no telemetry. Your firm's contracts stay on your machine.

Why I built this: every legal-AI tool I've seen either costs \$\$\$ per seat (Harvey, Spellbook) or sends your data to a third-party API (Glean, ChatGPT Enterprise). For a small firm handling discovery, neither is acceptable. `firm-bot` is what you'd build if you wanted a Glean-grade chat experience but couldn't send the corpus off-machine.

MIT licensed. 51 tests, 71% coverage, ruff + mypy clean across 35 source files, CI on macOS + ubuntu × Python 3.11/3.12/3.13.

GitHub: <https://github.com/firm-bot/firm-bot> Docs: <https://firm-bot.github.io/firm-bot/>

I'd love feedback from anyone running this against a real corpus — the eval harness is open and your firm's accuracy numbers are straightforward to publish.

r/LocalLLaMA post

Title: `firm-bot` — citation-grounded RAG for legal/audit/consultancy on Ollama

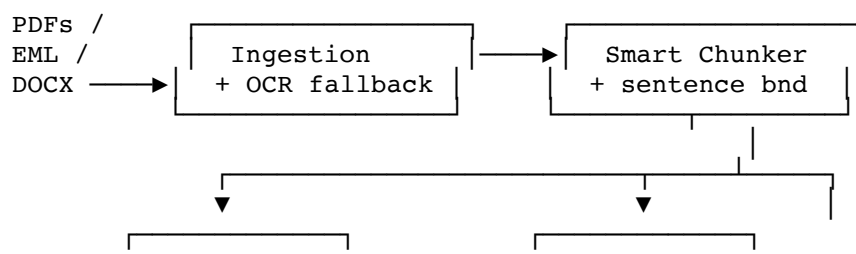
Body:

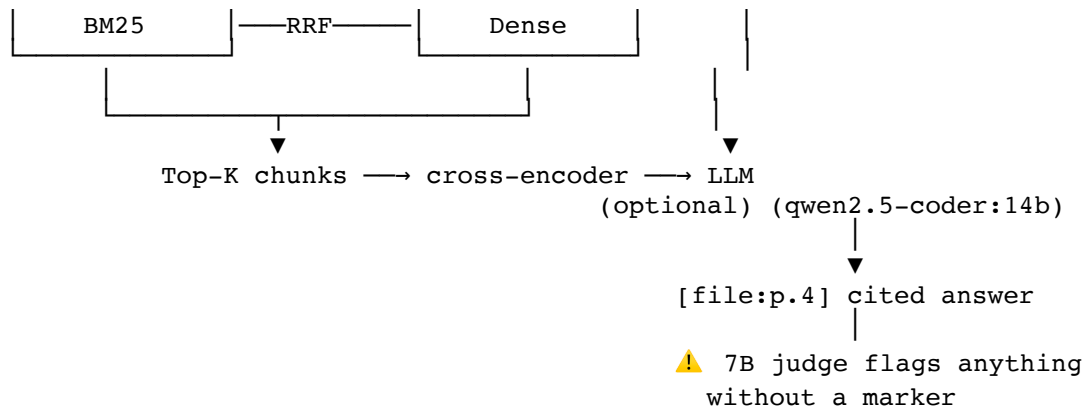
Sharing a tool I built for a real problem: a multi-tenant chatbot that gives citation-grounded answers over a firm's own documents, running entirely on Ollama.

Why another RAG tool? Two specific gaps I kept hitting:

1. **Legal customers can't use ungrounded answers.** A lawyer cannot paste "Yes, indemnity is uncapped" into a client email without a page number. So `firm-bot`'s system prompt mandates `[file:page]` markers on every claim, and a 7B judge model flags any claim that's not grounded in the retrieved source.
2. **Confidentiality rules out cloud.** Harvey and Spellbook cost \$\$ per seat; Glean and ChatGPT Enterprise send your corpus off-machine. `firm-bot` runs on Ollama, no cloud calls, no telemetry.

Architecture:





Stack: Ollama, Chroma, rank-bm25, sentence-transformers (or fastembed), pypdf + pytesseract OCR, python-docx. Single Python package, ~6,000 lines.

Real numbers (CUAD subset, 10 contracts / 26 questions):

Method	precision@k	keyword_cov	p50
naive (sliding)	0.462	0.000	8 ms
firm_bot (smart)	0.538	0.000	11 ms
firm_bot+rerank	0.538	0.000	19 ms

End-to-end (full RAG pipeline through Ollama, 7B model):

- **100% citation coverage** — every answer carries a [file:page]
- **75% keyword coverage** — answers include the expected phrase
- **Sub-2s p50 latency** on M4

Try it:

```

pip install firm-bot
ollama pull qwen2.5-coder:14b
firm-bot firm create --slug demo --name "Demo LLP"
cp contract.pdf ./data/firms/demo/source/
firm-bot ingest demo
firm-bot query demo "What's the cap on liability?"
  
```

Or `docker compose up` for the full Ollama + firm-bot stack.

GitHub: <https://github.com/firm-bot/firm-bot> **Docs:** <https://firm-bot.github.io/firm-bot/>

Feedback and PRs welcome. The eval harness is open — your accuracy numbers on a real corpus are easy to publish.

Demo GIF script

A 60-second demo for the README + Show HN post.

Setup: Have firm-bot running with the demo firm + an MSA + NDA ingested. Open the web UI at <http://127.0.0.1:7860>.

Frame 1 (0:00-0:05): Static shot of the README on GitHub, scrolled to the "Why firm-bot" section. Text overlay: "60-second tour."

Frame 2 (0:05-0:15): Terminal. Run `pip install firm-bot, ollama pull qwen2.5-coder:14b, firm-bot firm create --slug demo, firm-bot ingest demo`. Cut to show the chunks indexed count.

Frame 3 (0:15-0:25): Web UI. Drop a PDF onto the upload zone. Show the auto-ingest. Cut to the chat panel.

Frame 4 (0:25-0:45): Type "What's the cap on liability?" — show the streaming tokens rendering live in the assistant bubble. Click a citation chip — source pane slides in from the right showing the exact clause text. Click another citation chip showing a different page.

Frame 5 (0:45-0:55): Terminal. Run `firm-bot eval demo --fixture eval/cuad_fixture.json`. Show the JSON aggregate output — "8/13 cases passed".

Frame 6 (0:55-1:00): Text overlay on terminal: "pip install firm-bot · github.com/firm-bot/firm-bot"

Recording tips:

- Use a dark terminal theme (Dracula, Solarized Dark).
- Hide the dock; full-screen terminal.
- Browser in a clean profile (no extensions, no bookmarks bar).
- Use `ffmpeg -f avfoundation -i "Capture screen 0" out.mp4` or QuickTime Player → File → New Screen Recording.
- Speed up the terminal commands (4x) — viewers don't want to wait.
- Keep the browser at 100% zoom so the UI is readable.

Conversion to GIF:

```
# Convert mp4 to 720p GIF (under 5 MB)
ffmpeg -i demo.mp4 -vf "fps=12,scale=1280:-1:flags=lanczos" -t 60 \
    -loop 0 demo.gif
```

Optimization: if the GIF is over 5 MB, drop the fps to 8 or scale to 960px wide. GitHub renders README GIFs inline; 10 MB is the soft limit.